Plataforma Distribuida de Gestión de Pedidos
Reto Microservicios - Ingeniería de Software – Autora: Gloria Gutiérrez

1. Alcance del Proyecto

1.1 Alcance General

Diseñar e implementar una solución distribuida basada en arquitectura de microservicios utilizando **Spring Boot y Angular**, que permita gestionar autenticación, catálogo de productos y pedidos, integrando comunicación síncrona (REST) y asíncrona (RabbitMQ), con seguridad basada en JWT, trazabilidad distribuída y persistencia independiente por servicio.

La solución deberá aplicar principios modernos de arquitectura distribuída, garantizando desacoplamiento, consistencia eventual y observbilidad.

El sistema deberá:

- Implementar API Gateway como punto único de entrada.
- Garantizar separación de responsabilidades por microservicio.
- Aplicar base de datos por servicio.
- Implementar comunicación por eventos.
- Incluir trazabilidad con Correlation ID.
- Incorporar seguridad basada en JWT.
- Exponer health checks.

1.2 Alcance Funcional

El sistema debe permitir las siguientes funcionalidades:

1. Gestión de Autenticación de usuarios (Registro (opcional) o carga inicial de usuarios. | Inicio de sesión mediante credenciales. | Generación de token JWT | Gestión de roles (ADMIN / CUSTOMER). | Protección de endpoints mediante autorización.)
2. Gestión de Catálogo (Consulta de catálogo de productos. | Consulta del detalle del producto | Visualización de stock disponible | end point para validación del Stock (check-stock) | Ajuste de inventario mediante consumo de eventos.)
3. Gestión de pedidos (Creación de pedidos con múltiples productos. | Validación previa de disponibilidad de stock. | Persistencia del pedido con estado inicial CREATED, consulta de pedidos por usuario. | Cambio de estado del pedido. | Cancelación del pedido.)
4. Gestión de eventos (Publicación de evento order.created.| Publicación de evento order.cancelled. | Inclusión de eventId y correlationId en cada evento.| Consumo idempotente de eventos. | Actualización automática de inventario mediante eventos.)
5. Manejo de errores (Respuesta 401 ante token inválido o ausente. | Respuesta 403 ante rol no autorizado. | Respuesta 409 cuando no exista stock suficiente. Manejo de errores controlados en todos los servicios.)

1.3 Alcance Técnico

El proyecto debe incluir:

1. Arquitectura (API Gateway como punto único de entrada. | Tres microservicios independientes: Auth Service, Catalog Service y Orders Service | Separación por responsabilidad (Single Responsibility Principle). | Comunicación REST entre servicios.| Comunicación asíncrona mediante RabbitMQ. | Base de datos independiente por microservicio.)
2. Seguridad (Autenticación basada en JWT. | Validación de roles en endpoints. | Propagación del header Authorization. | Protección de rutas sensibles.)
3. Observabilidad (Generación y propagación de X-Correlation-Id. | Inclusión de correlationId en eventos. |Logs estructurados por servicio. | Exposición de /actuator/health.)
4. Comunicación (REST para validación síncrona de stock.| RabbitMQ para eventos asíncronos.| Manejo idempotente en consumidores. | Consistencia eventual en actualización de inventario.)
5. Persistencia (PostgreSQL por microservicio. | Integridad transaccional interna por servicio. | Tabla para control de eventos procesados. | Modelado independiente por dominio.)
6. Infraestructura (Ejecución en contenedores Docker. | Configuración por variables de entorno. | Red interna para comunicación entre servicios. | Separación lógica por capas (Application, Messaging, Infrastructure).)

◆1.4 Fuera del Alcance

1. Orquestación distribuida tipo Saga avanzada.
2. Despliegue en nube pública.
3. Monitoreo con Prometheus/Grafana (solo Actuator básico).
4. Balanceo de carga productivo.
5. No se incluirán tecnologías como IA o Blockchain.

2. Objetivo General

Diseñar e implementar una arquitectura de microservicios funcional, desacoplada y observable que permita gestionar pedidos y stock mediante comunicación síncrona y asíncrona, aplicando principios de diseño modernos y buenas prácticas empresariales.

2.1 Objetivos Específicos

◆ **Arquitectura**

1. Implementar API Gateway como punto único de entrada.
2. Separar el sistema en microservicios independientes.
3. Aplicar el principio de base de datos por servicio.
4. Garantizar bajo acoplamiento entre servicios.

◆ **Seguridad**

5. Implementar autenticación basada en JWT.
6. Validar roles en endpoints protegidos.
7. Propagar Authorization y Correlation ID en todas las solicitudes.

◆ **Comunicación**

8. Implementar validación síncrona de stock vía REST.
9. Implementar publicación de eventos en RabbitMQ.
10. Implementar consumo idempotente de eventos.
11. Aplicar consistencia eventual.

◆ **Observabilidad**

12. Implementar propagación de X-Correlation-Id.
13. Registrar trazas completas en logs.
14. Exponer endpoint /actuator/health en todos los servicios.

◆ **Persistencia**

15. Diseñar modelo de datos independiente por servicio.
16. Implementar control de estados del pedido.
17. Garantizar integridad transaccional interna en cada servicio.

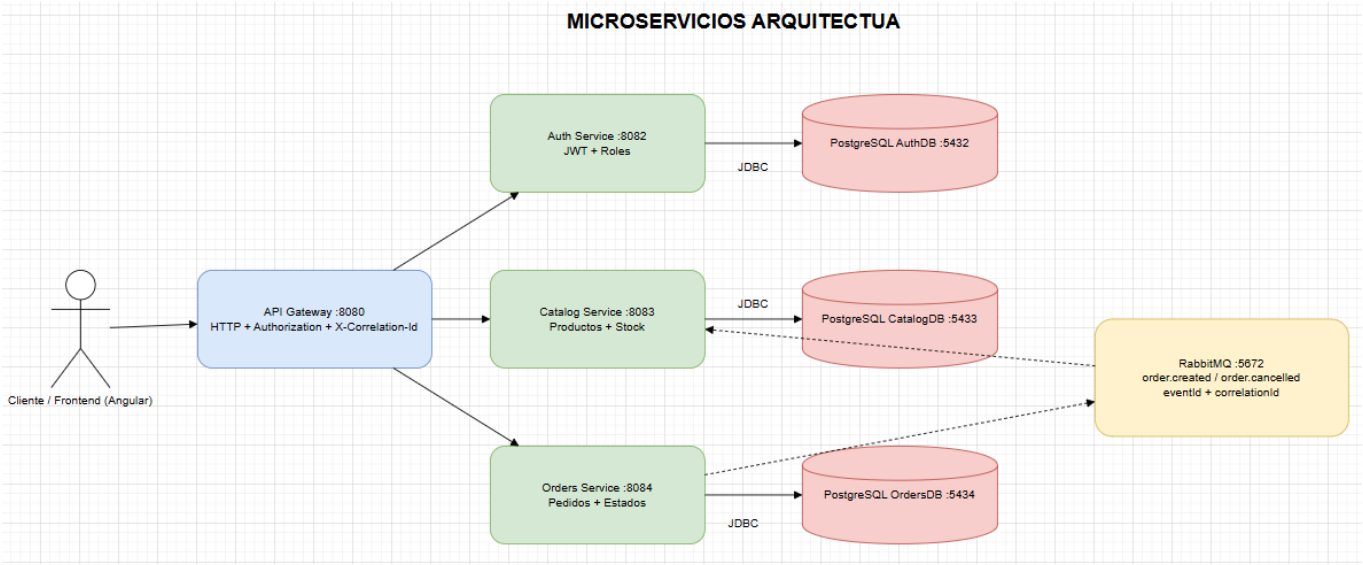
◆ **Calidad**

18. Manejar errores HTTP adecuados (200, 401, 403, 409).
19. Validar escenarios sin stock.
20. Documentar la arquitectura y flujos.

3. Arquitectura de la Solución

El proyecto utiliza una arquitectura de microservicios basada en los siguientes componentes: API Gateway, Auth Service, Catalog Service, Orders Service, RabbitMQ y bases de datos separadas.

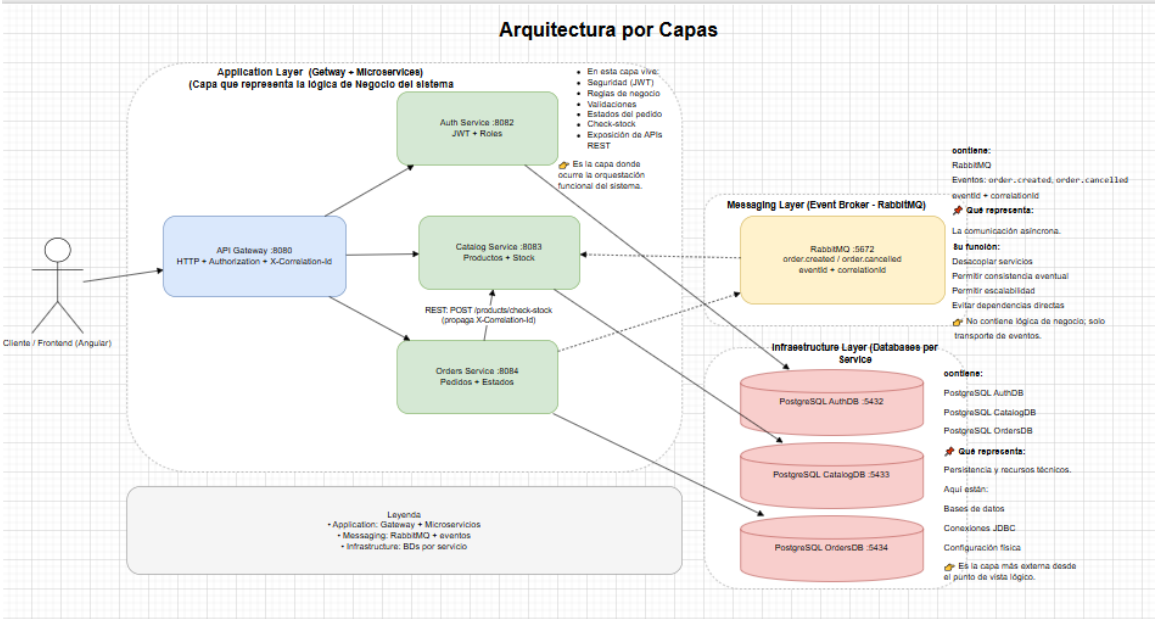
3.1 Arquitectura a Alto Nivel



1. Usuario realiza login en Angular.
2. Angular envía credenciales al Gateway.
3. Gateway redirige a Auth Service.
4. Auth genera JWT.
5. Angular envía peticiones con Authorization: Bearer.
6. Orders valida stock vía REST contra Catalog.
7. Orders publica evento en RabbitMQ.

8. Catalog consume evento y ajusta stock.
9. Todos los servicios exponen /actuator/health.
10. Gateway inyecta X-Correlation-Id para trazabilidad.

3.2 Arquitectura por Capas



El sistema está diseñado bajo una arquitectura de microservicios dividida en tres capas claramente diferenciadas: Application Layer, Messaging Layer e Infrastructure Layer.
El objetivo fue construir un sistema desacoplado, seguro, observable y preparado para escalar.

3.2.1 Capa 1 – Application Layer

En la Application Layer se encuentran el API Gateway y los microservicios: Auth, Catalog y Orders.

◆ API Gateway

- Es el punto único de entrada.
- Centraliza rutas.
- Propaga el X-Correlation-Id.
- Maneja autenticación basada en JWT.

El frontend nunca llama directamente a los microservicios, solo al Gateway.

◆ Auth Service

- Gestiona login.
- Genera JWT.
- Administra roles.

◆ Catalog Service

- Gestiona productos y stock.
- Valida disponibilidad.

◆ Orders Service

- Gestiona pedidos y estados.
- Orquesta el flujo de negocio.

Cada servicio tiene responsabilidad única y no comparte su base de datos.

3.2.2 Capa 2 – Messaging Layer

La Messaging Layer está compuesta por RabbitMQ, que implementa comunicación asíncrona.

Cuando se crea un pedido:

- Orders publica order.created
- Incluye eventId y correlationId
- Catalog consume el evento

- Descuenta stock

Esto nos permite desacoplar servicios y aplicar consistencia eventual.

◆ Idempotencia

- Catalog guarda eventId
- Si el evento se repite, no procesa dos veces

Eso evita inconsistencias en escenarios reales de reintentos.

3.2.3 Capa 3 – Infrastructure Layer

Cada microservicio tiene su propia base de datos:

- AuthDB
- CatalogDB
- OrdersDB

Esto garantiza autonomía, bajo acoplamiento y escalabilidad independiente. Ningún servicio accede a la base de datos de otro.

3.2.4 Observabilidad

La arquitectura incorpora trazabilidad distribuida.

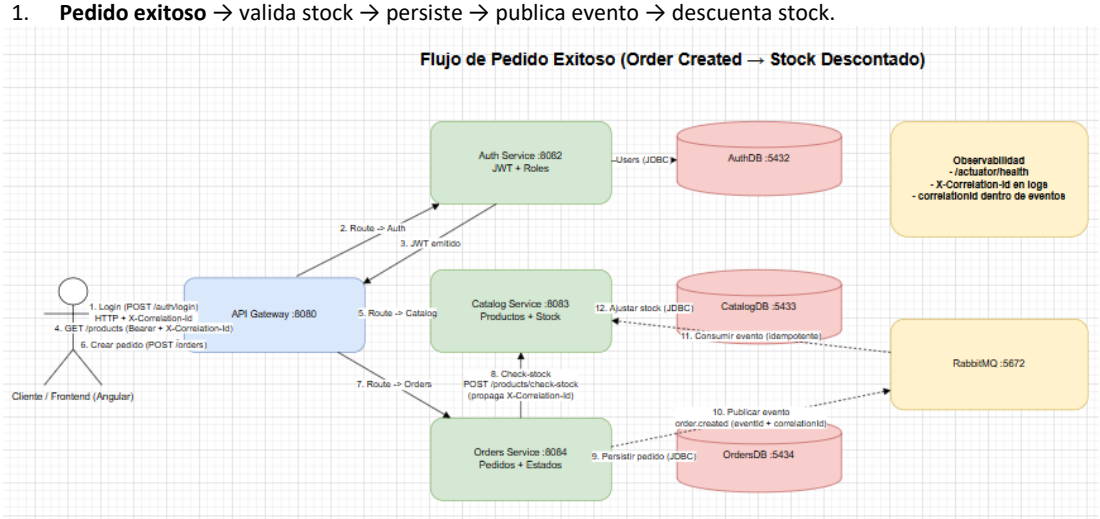
- Gateway genera X-Correlation-Id
- Se propaga en REST
- Se incluye en eventos
- Se registra en logs
- Todos los servicios exponen /actuator/health

Esto permite rastrear una transacción completa a través de todo el sistema.

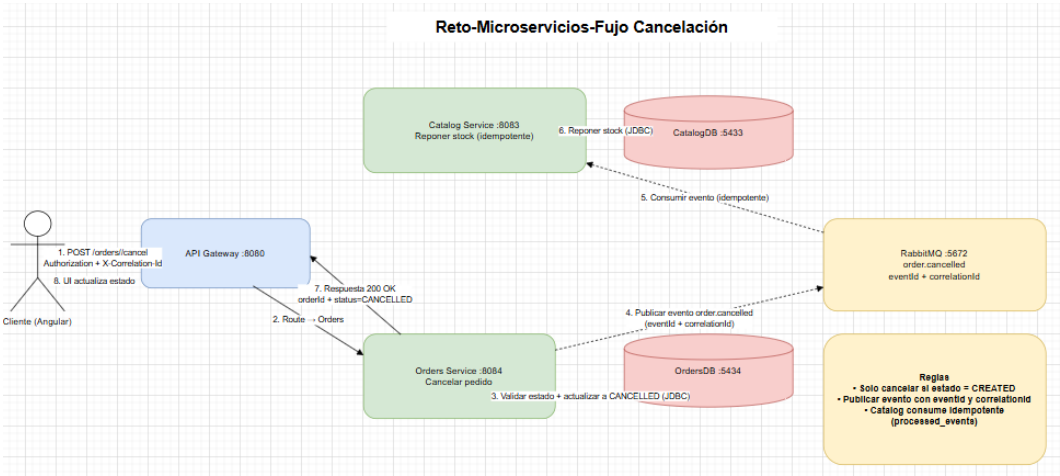
3.2.5 Flujos Principales

Tenemos tres escenarios:

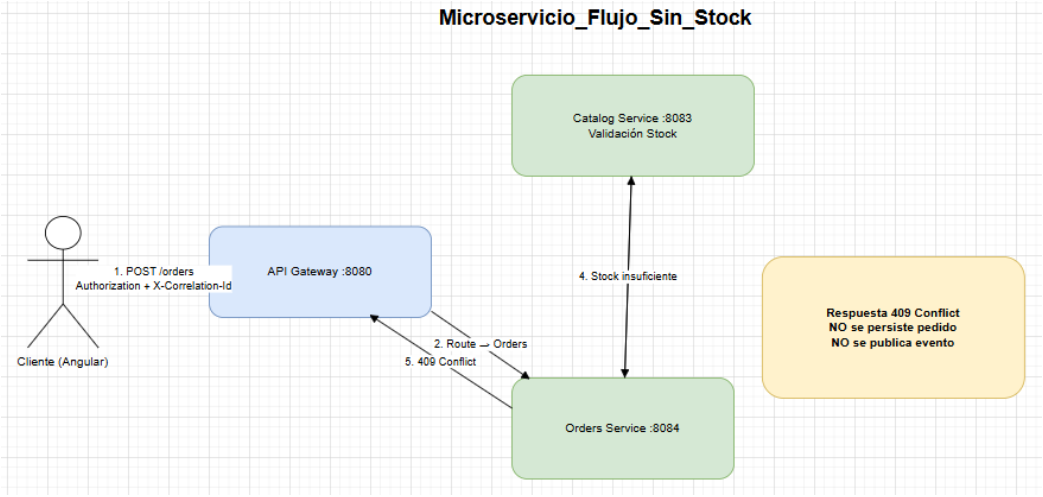
1. **Pedido exitoso** → valida stock → persiste → publica evento → descuenta stock.



2. **Cancelación** → cambia estado → publica evento → repone stock.



3. **Sin stock** → retorna 409 → no persiste ni publica evento.



Cada flujo respeta separación de responsabilidades, persistencia independiente y comunicación desacoplada.

4. Aspectos técnicos

4.1 Arquitectura de Microservicios

La arquitectura de microservicios propone dividir un sistema en componentes autónomos que poseen su propia lógica de negocio y base de datos. Este enfoque mejora escalabilidad, resiliencia y mantenibilidad.

4.2 Uso de RabbitMQ

- RabbitMQ es un broker de mensajería que permite comunicación asíncrona entre servicios. En el proyecto, Orders publica eventos (OrderCreated, OrderCancelled) y Catalog los consume para ajustar el stock. Esto reduce el acoplamiento y permite consistencia eventual.
- RabbitMQ implementa el patrón Publish/Subscribe. Orders publica eventos en un exchange tipo topic. Catalog se suscribe mediante routing keys específicas. Se implementa idempotencia con una tabla processed_events para evitar duplicidad.

En este proyecto:
- Orders publica eventos: order.created y order.cancelled.
- Catalog consume eventos.
- Se usa un exchange tipo topic: orders.exchange.
- Se implementa idempotencia con tabla processed_events.

Beneficios:
- Desacoplamiento.
- Escalabilidad.
- Resiliencia.
- Consistencia eventual.

4.3 Observabilidad

La observabilidad permite entender el comportamiento del sistema distribuido. Se implementa mediante:
- Actuator (/health Checks) en cada servicio:
  - Endpoint: /actuator/health
  - Permite verificar disponibilidad del servicio.
- Correlation ID (X-Correlation-Id)
  - Header: X-Correlation-Id
  - Generado en Gateway.
  - Permite rastrear una transacción completa entre servicios.
  - Permite rastrear una petición a través de múltiples microservicios.
- Logs estructurados
  - Cada servicio registra eventos importantes.
  - Permite seguimiento de flujo.
- Monitoreo básico de RabbitMQ
  - UI en puerto 15672
  - Permite visualizar colas, exchanges y mensajes.

Estos mecanismos permiten diagnosticar fallos en sistemas distribuidos.

4.4. Idempotencia

Si ejecutas la misma operación varias veces con la misma información, el resultado final es el mismo que si la ejecutaras una sola vez. No importa si se repite. El estado final no cambia más allá de la primera ejecución válida.

Por qué es crítica en microservicios

En sistemas distribuidos:

- Las redes fallan
- Los brokers reintentan

Si no implementas idempotencia:

- El sistema queda inconsistente
- Los datos se corrompen
- No hay confiabilidad

4.5 Seguridad con JWT

- JWT permite autenticación stateless. Los roles (ADMIN, CUSTOMER) permiten control de acceso basado en políticas.
- El sistema utiliza JSON Web Tokens para autenticación y autorización. Auth genera el token, el Gateway lo valida y los servicios confían en su contenido.

4.6 Consistencia Eventual

El uso de eventos introduce el concepto de consistencia eventual, donde los servicios alcanzan coherencia de datos tras propagación del evento.

4.7 REST

REST (Representational State Transfer) es un estilo de arquitectura para construir APIs que usan HTTP.

- No es una tecnología.
- No es una librería.
- Es una forma de diseñar servicios web.

REST significa: “Un servicio expone recursos a través de URLs y se interactúa con ellos usando métodos HTTP.”

En REST usamos:

Método	Significado
GET	Consultar
POST	Crear
PUT	Actualizar
DELETE	Eliminar

Ejemplo en el proyecto

- Cuando Orders valida stock:
  GET /catalog/check-stock?productId=1&quantity=3
  Orders hace una llamada REST a Catalog.
- Catalog responde:
  ```
  {
    "available": true
  }
  ```
- Eso es comunicación síncrona.
- Orders espera la respuesta antes de continuar.

REST es:

- Stateless (no guarda sesión en servidor)
- Basado en HTTP
- Orientado a recursos
- Síncrono (request/response)

¿Por qué usamos REST en este proyecto?

- Necesitamos validación inmediata de stock.
- Orders no puede crear un pedido si no hay stock.
- Necesitamos respuesta en tiempo real.

4.8 CorrelationId

CorrelationId es un identificador único que viaja a través de todos los servicios durante una misma operación.
Sirve para rastrear una solicitud en un sistema distribuido.

Problema que resuelve

En microservicios, una acción puede pasar por:

- Gateway
- Orders
- Catalog

- RabbitMQ

Cada uno genera logs distintos.
Sin CorrelationId no puedes saber qué logs pertenecen a la misma solicitud.

Ejemplo práctico

- Un usuario crea un pedido.
- Gateway genera: X-Correlation-Id: abc-123
- Ese mismo ID:
  - Se envía en la llamada REST a Catalog
  - Se incluye en el evento RabbitMQ
  - Se registra en los logs de todos los servicios

Entonces puedes buscar: abc-123 Y ver todo el flujo completo.

En el proyecto

Se implementa así:

- Gateway genera el ID si no existe.
- Se agrega al header HTTP.
- Se incluye en el mensaje del evento.
- Se imprime en logs.

Concepto clave

CorrelationId es una forma básica de trazabilidad distribuida. Es una versión simple de herramientas profesionales como:

- OpenTelemetry
- Zipkin
- Jaeger

4.9 Health Check

Un Health Check es un endpoint que permite saber si un servicio está funcionando correctamente. En Spring Boot se usa: /actuator/health
Puede verificar:

- Si la aplicación está viva
- Si la base de datos responde
- Si RabbitMQ está conectado

Ejemplo de respuesta

```
{
  "status": "UP"
}
```

Si algo falla:

```
{
  "status": "DOWN"
}
```

¿Por qué es importante En arquitectura distribuida?

- Un servicio puede estar encendido pero no operativo.
- Puede haber problemas de conexión a DB.
- Puede fallar el broker.

Health Check permite monitorear eso.

En tu Proyecto Cada microservicio debe exponer: /actuator/health

Esto permite:

- Verificar disponibilidad
- Simular monitoreo real
- Prepararlos para entornos productivos

5. Planificación del Proyecto

A continuación se presenta la distribución de actividades por 3 Sprint, cada Sprint es de 1 semana. Podrán realizar los ajustes o gregar ëpicas, HU, Criterios de Aceptación y actividades si lo requieren

SPRINT	EPICA	HU	CRITERIOS DE ACEPTACION	SPRING BACKLOG - ACTIVIDADES
Sprint 1 – Arquitectura Base y Seguridad	**Onboarding Project**	**Onboarding Project**	**Onboarding Project**	Presentación del reto Revisión de arquitectura Explicación de capas Setup entorno (Docker, PostgreSQL, RabbitMQ)
	Épica 1: Infraestructura Base	**HU1 – Configurar Gateway** Como desarrollador Quiero un API Gateway funcional Para enrutar solicitudes a microservicios	**Criterios de aceptación:** Ruta a Auth funciona Ruta a Catalog funciona Ruta a Orders funciona	Crear API Gateway Configuración básica de rutas Verificación de conexión entre servicios
	Épica 2: Autenticación	**HU2 – Login JWT** Como usuario Quiero autenticarme Para recibir un token válido	**Criterios de aceptación:** Genera JWT firmado Token expira 401 si credenciales inválidas	Crear Auth Service Configuración JWT Endpoint /auth/login Protección de rutas Pruebas con token válido e inválido Manejo de 401 / 403

SPRINT	EPICA	HU	CRITERIOS DE ACEPTACION	SPRING BACKLOG
Sprint 2 – Dominio y Comunicación - Eventos	**Épica 3: Catálogo de Servicios**	**HU3 – Crear Catálogo Services** Como usuario Quiero crear el catálogo de productos para registrar **inventario** y stock	**Criterios de Aceptación:** Crear productos Modificar productos Gestionar cantidades disponibles Descontar stock cuando se confirma un pedido Reponer stock cuando se cancela un pedido	Crear Catalog Service CRUD productos Base de datos independiente
	Épica 4: Gestión de Pedidos	**HU4 – Crear Pedido** Como usuario Quiero crear pedido Para registrar una compra	**Criterios de Aceptación** Persiste en OrdersDB Estado inicial CREATED Responde 200	Crear Orders Service Modelo de pedido y estados Persistencia en OrdersDB
		HU5 – Validar Stock Como Orders Service Quiero validar stock vía REST Para evitar pedidos inválidos	**Criterios de aceptación:** Llama a check-stock Propaga CorrelationId Responde 409 si no hay stock	Implementar check-stock REST Integración Orders → Catalog Configurar RabbitMQ
	Épica 5: Eventos	**HU6 – Publicar Evento order.created** Como Orders Service Quiero publicar evento Para notificar al sistema	**Criterios de aceptación:** Incluye eventId Incluye correlationId Se publica en RabbitMQ	Publicación de order.created
		HU7 – Consumir Evento Idempotente Como Catalog Quiero consumir eventos Para actualizar stock sin duplicar	**Criterios de aceptación:** Guarda eventId No descuenta dos veces	Consumo idempotente en Catalog Tabla processed_events

SPRINT	EPICA	HU	CRITERIOS DE ACEPTACION	SPRING BACKLOG
Sprint 3 – Calidad, Cancelación y Observabilidad	**Épica 6: Cancelación**	**HU8 – Cancelar Pedido** Como usuario Quiero cancelar pedido Para revertir la compra	**Criterios de aceptación:** Cambia estado a CANCELLED Publica order.cancelled Reposición de stock	Implementar cancelación de pedido
				Publicar order.cancelled
				Reposición de stock
	Épica 7: Observabilidad	**HU9 – Correlation ID** Como sistema Quiero propagar CorrelationId Para trazabilidad distribuida	**Criterios de aceptación:** Gateway lo genera Se propaga en REST Se incluye en eventos Se registra en logs	Implementar X-Correlation-Id
				Propagación en REST
				Incluir en eventos
				Logs estructurados
		HU10 – Health Check Como administrador Quiero verificar estado de servicios Para monitorear disponibilidad	**Criterios de aceptación:** /actuator/health responde OK Todos los servicios exponen endpoint	Actuator en todos los servicios
				Manejo de errores 409 (sin stock)
				Validaciones completas
				Integración con Angular
				Flujo completo E2E
				Pruebas finales
				Documentación

GUÍA DE ORIENTACIÓN PARA LA EJECUCIÓN DEL PROYECTO

Stack recomendado:

Capa	Tecnología
Backend	Spring Boot 3.2 + Java 21
Seguridad	Spring Security + JWT
Base de datos	PostgreSQL
Comunicación	REST
Mensajería (más adelante)	RabbitMQ
Frontend	Angular
Infraestructura	Docker Compose

Arquitectura técnica sugerida

</> Código
```
Angular (4200)
      ↓
API Gateway (8080)
      ↓
-------------------------------
Auth Service (8081)
Catalog Service (8082)
Order Service (8083)
-------------------------------
PostgreSQL
RabbitMQ (Sprint 2+)
```

Sprint 1 — Guía técnica paso a paso (para estudiantes)

Recomendación: Trabajar 100% backend en este sprint, Dront y RabbitMQ después.

- Microservicios funcionando
- JWT funcionando
- Postman como cliente

Dejar para el Sprint 2 Angular consume Auth + Catalog, Si no hacen eso, el 70% del tiempo lo perderán peleando con CORS y no aprendiendo arquitectura.

0) Checklist de herramientas (antes de tocar código)

1. Instalar:
    - Java 21 (LTS)
    - Maven 3.9 +
    - Docker Desktop
    - Postman
    - IDE: IntelliJ / VSCode
    - Spring Boot 3.2 o superior
    - Angular (17 o 18)
    - PostgreSQL 16

En el pom.xml
```
<properties>
  <java.version>21</java.version>
</properties>
```

2. Verificar desde consola:

</>Bash
```
java -version  debe mostrar openjdk 21
mvn -version
docker --version
docker compose version
```
✅ Criterio: si estos comandos responden, el entorno está listo.

Configuración recomendada para java 21

application.yml

```
server:
    port:8081
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/authdb
    username: reto
    password: reto
  jpa:
    hibernate:
        ddl-auto: update
    properties:
        hibernate:
          format_sql: true
security:
jwt:
# IMPORTANTE: mínimo 32 chars para HS256
secret: "CAMBIAR_ESTA_CLAVE_SUPER_LARGA_DE_32_CHARS_MINIMO_!!"
expiration-minutes: 15
```

Problema: Asegurarse que está compilando la versión correcta de java

---

Angular + Microservicios

Angular SOLO habla con:

`http://localhost:8080`

Nunca directo a 8081, 8082, etc.

Ejemplo servicio Angular:

```
login(data: any) {
  return this.http.post('http://localhost:8080/auth/login', data);
}
```

Guardar token en:

- memory (mejor para clase)
- o localStorage (más práctico)

Luego enviarlo en header:

```
Authorization: Bearer <token>
```

Problema de Angular bloqueado por CORS

```
Solución rápida en backend:
@Bean
public WebMvcConfigurer corsConfigurer() {
    return new WebMvcConfigurer() {
        @Override
        public void addCorsMappings(CorsRegistry registry) {
            registry.addMapping("/**")
                    .allowedOrigins("http://localhost:4200")
                    .allowedMethods("*");
        }
    };
}
```

Problema JWT mal implementado

Deben:

- Firmar con secret fuerte
- Validar expiración
- No confiar en datos del frontend

PostgreSQL — configuración profesional mínima

En docker-compose:

```
postgres:
  image: postgres:16
  environment:
    POSTGRES_USER: reto
    POSTGRES_PASSWORD: reto
    POSTGRES_DB: authdb
  ports:
    - "5432:5432"
```

Para producción futura:

- Nunca exponer puerto 5432.
- Usar variables de entorno.
- Separar bases por servicio (authdb, catalogdb, orderdb).

---

1) Estructura del repositorio (monorepo recomendado)

Crear una carpeta raíz, ejemplo:

</> Código

```
reto-microservicios/
  docker-compose.yml
  README.md
  services/
    api-gateway/
      pom.xml
      src/main/java/com/reto/gateway/
        ApiGatewayApplication.java
        security/
          JwtGatewayFilter.java          (Sprint 1 o 2)
      src/main/resources/
        application.yml
    auth-service/
      pom.xml
      src/main/java/com/reto/auth/
        AuthServiceApplication.java
        config/
          SecurityConfig.java
          PasswordConfig.java
        controller/
          AuthController.java
        dto/
          RegisterRequest.java
          LoginRequest.java
          TokenResponse.java
        entity/
          UserEntity.java
          Role.java
        repository/
          UserRepository.java
        security/
          JwtService.java
          JwtAuthFilter.java
        service/
          AuthService.java
        exception/
          ApiExceptionHandler.java        (opcional Sprint 1)
      src/main/resources/
        application.yml
    catalog-service/
      pom.xml
      src/main/java/com/reto/catalog/
        CatalogServiceApplication.java
        controller/
          PingController.java
      src/main/resources/application.yml
    order-service/
      pom.xml
      src/main/java/com/reto/order/
        OrderServiceApplication.java
        controller/
          PingController.java
      src/main/resources/application.yml
```

Reglas para que sea microservicios:

- Cada servicio con su `pom.xml` y su `application.yml`
- Cada servicio en su **puerto**
- Cada servicio con su **BD propia** (authdb, catalogdb, orderdb) — en Sprint 1 mínimo authdb

---

2) Setup entorno con Docker (PostgreSQL + RabbitMQ)

Crear `docker-compose.yml` en la raíz:

</> YAML
```yaml
services:
  postgres:
    image: postgres:16
    container_name: reto_postgres
    environment:
      POSTGRES_USER: reto
      POSTGRES_PASSWORD: reto
      POSTGRES_DB: authdb
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  rabbitmq:
    image: rabbitmq:3-management
    container_name: reto_rabbitmq
    ports:
      - "5672:5672"
      - "15672:15672" # UI
    environment:
      RABBITMQ_DEFAULT_USER: reto
      RABBITMQ_DEFAULT_PASS: reto

volumes:
  pgdata:
```

Levantar:
</>Bash
```bash
docker compose up -d
docker ps
```
✅ Criterios:

- PostgreSQL arriba en **5432**
- RabbitMQ arriba en **5672**
- UI RabbitMQ abre en navegador (http://localhost:15672) con user/pass `reto/reto`

Nota útil: RabbitMQ en Sprint 1 puede quedar "arriba y listo", aunque no lo usen aún. Así evitas que Sprint 2 se vuelva infierno.

---

3) Crear servicios base (Spring Initializr)

3.1 API Gateway (Spring Cloud Gateway)

Proyecto: `services/api-gateway`
Dependencias:

- Spring WebFlux
- Spring Cloud Gateway
- Spring Boot Actuator

Puerto sugerido: **8080**
✅ Criterio: arranca y responde `GET /actuator/health` → `UP`.

3.2 Auth Service (Spring Security + JWT + PostgreSQL)

Proyecto: `services/auth-service`
Dependencias:

- Spring Web
- Spring Security
- Spring Data JPA
- PostgreSQL Driver

- Validation
- Spring Boot Actuator

Puerto sugerido: **8081**

✅ Criterio: arranca y conecta a Postgres.

3.3 Catalog y Orders (STUB por ahora)

En Sprint 1 NO los implementen completos. Solo:

- un endpoint "ping" para probar routing.

catalog-service puerto **8082**
order-service puerto **8083**
Endpoints mínimos:

- `GET /ping` → "catalog ok"
- `GET /ping` → "order ok"

✅ Criterio: gateway puede enrutar a `/catalog/ping` y `/orders/ping`.

---

4) Configurar puertos y nombres (application.yml)

4.1 auth-service: `application.yml`

```
server:
  port: 8081

spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/authdb
    username: reto
    password: reto
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

✅ Criterio: en logs debe verse que Hibernate crea tablas sin error.

---

5) HU1 — Configurar Gateway + rutas (lo de tu tabla)

En `api-gateway/src/main/resources/application.yml`:

```
server:
  port: 8080

spring:
  cloud:
    gateway:
      routes:
        - id: auth
          uri: http://localhost:8081
          predicates:
            - Path=/auth/**

        - id: catalog
          uri: http://localhost:8082
          predicates:
            - Path=/catalog/**
          filters:
            - StripPrefix=1

        - id: orders
          uri: http://localhost:8083
          predicates:
            - Path=/orders/**
          filters:
            - StripPrefix=1
```

✅ Criterios de aceptación de HU1 (los de tu imagen):

- Ruta a **Auth** funciona: `POST http://localhost:8080/auth/login` llega a auth-service.
- Ruta a **Catalog** funciona: `GET http://localhost:8080/catalog/ping`
- Ruta a **Orders** funciona: `GET http://localhost:8080/orders/ping`

Detalle didáctico importante:
Con `StripPrefix=1`, `/catalog/ping` se convierte en `/ping` al llegar al servicio, que es lo que quieres si el servicio no conoce el prefijo.

---

6) HU2 — Login JWT (implementación mínima pero correcta)

6.1 Entidad y repositorio

Entidad `User`:

- id
- email (único)
- password (hash)
- role (ADMIN/CUSTOMER)

Repositorio `UserRepository` con `findByEmail`.

✅ Criterio: puedes registrar un usuario y se guarda.

6.2 Registro y login (2 endpoints)

- `POST /auth/register` (email, password, role opcional)
- `POST /auth/login` (email, password) → retorna token JWT

Reglas mínimas:

- Password se guarda con **BCrypt**
- Si credenciales inválidas → **401**
- Token tiene expiración (ej: 15 min) → si está vencido, no sirve

✅ Criterios de aceptación HU2 (los de tu imagen):

- Genera JWT **firmado**
- Token **expira**
- `401` si credenciales inválidas

6.3 JWT "sin humo": qué deben tener sí o sí

- Un **secret** (clave) en `application.yml`:

```
security:
  jwt:
    secret: "CAMBIAR_ESTA_CLAVE_SUPER_LARGA_QUE_NO_SEA_123"
    expiration-minutes: 15
```

- El token debe incluir:
  - `sub` (email o userId)
  - `role`
  - `iat`, `exp`

---

7) Protección de rutas (401 / 403) — lo que más se les enreda

Tienes 2 opciones. Para estudiantes nuevos, recomiendo esta:

Opción A (simple y clara): cada microservicio valida JWT

- En Sprint 1, puedes dejarlo solo en auth-service + gateway sin filtro.
- En Sprint 2, se replica validación en order/catalog.

Pero tu tabla dice "Protección de rutas" ya en Sprint 1, entonces:

Opción B (sprint1-friendly): Gateway valida JWT (recomendado)

Implementar un **GlobalFilter** en el gateway que:

- Deje pasar libre: `/auth/**`
- Requiera token para `/orders/**`
- Opcional: deje libre GET `/catalog/**` y proteja POST/PUT/DELETE (más real)

Validaciones:

- Si NO hay Authorization: Bearer → **401**
- Si token inválido/expirado → **401**
- Si tiene token pero rol no permitido → **403**

✅ Criterio: ustedes pueden demostrar 3 casos con Postman:

1. Sin token a `/orders/ping` → 401
2. Token malo a `/orders/ping` → 401
3. Token válido pero rol incorrecto a ruta admin → 403

---

8) Verificación de conexión entre servicios (pruebas mínimas)

Prueba 1: salud

- `GET http://localhost:8080/actuator/health`
- `GET http://localhost:8081/actuator/health` etc.

Prueba 2: routing

- `GET http://localhost:8080/catalog/ping`
- `GET http://localhost:8080/orders/ping`

Prueba 3: auth

1. `POST http://localhost:8080/auth/register`
2. `POST http://localhost:8080/auth/login` → copia token
3. `GET http://localhost:8080/orders/ping` con header:
    - `Authorization: Bearer <token>`

✅ Criterio: todo esto queda grabado en una colección Postman y es "demo Sprint 1".

---

9) Entregables Sprint 1

Entregar los siguientes artefactos:

1. Repositorio con estructura `services/`
2. `docker-compose.yml` funcional (Postgres + RabbitMQ)
3. API Gateway con rutas a auth/catalog/orders
4. Auth Service con:
    - `/auth/register`
    - `/auth/login` → JWT firmado con exp
5. Pruebas postman
    - prueba sin token → 401
    - token inválido → 401
    - token válido → 200
6. Colección Postman exportada (`.json`)

Errores típicos (para que te adelantes y los frenes)

- **No arranca Postgres**: puerto 5432 ocupado (cerrar otro Postgres o cambiar puerto).
- **Gateway no enruta**: se olvidan de `StripPrefix=1` o el Path está mal.
- **JWT "funciona" pero no expira**: no están poniendo `exp` o no lo validan.
- **401 vs 403 mal**: 401 es "no autenticado"; 403 es "autenticado pero no autorizado".
- **Password en texto plano**: prohibido. Siempre BCrypt.